JASPY Web

Technical Documentation & Main Functions List

Project Overview

JASPY Web is a web application developed using **Python (Flask)** for the server side, and **JavaScript, CSS, and HTML** for the client side.

The core objective is to conceal direct HTML structures. Instead, JavaScript and CSS manage, create, and build all pages dynamically via functions, while the Python backend automatically handles route generation and new pages.

Python **JavaScript** **CSS** **HTML**

1. First Section: head.js

Functions responsible for building and setting elements within the page header (`<head>`):

- `title(a)`: Creates and sets the page title (`<title>`).
- `h_script(a): Imports additional JavaScript script files into the head.`
- `link(a): Imports CSS stylesheet files.`
- `icon(a): Sets the browser tab icon (favicon).`

2. Second Section: main.js (Elements & Structure Creation)

Element Creation Functions

- `Create_Element_simple_after_root(a)`: Creates an HTML element and appends it directly inside the `root` container.
- `Create_other_Element_simple(tg, isa, a)`: Creates an element inside a specified tag using its index.
- `Create_Element_text_after_root(a, b, c, d)`: Creates an element with text content and an attribute, then appends it to `root`.
- `Create_Element_after_root(a, c, d)`: Creates an element with an attribute inside `root`.
- `Create_other_Element_text(id, a, b, c, d)`: Creates an element with text and an attribute inside a specified element ID.
- `Create_other_Element(id, a, c, d)`: Creates a new element with an attribute inside a specified ID.
- `Create_other_Element_ByTag(tg, isa, a)`: Creates an element inside a tag and index.

Attributes & Control Functions

- `Set_Attribut(id, c, d)`: Modifies or sets an attribute on an element specified by ID.
- `Set_Attribut_ByTag(tg, isa, c, d)`: Modifies attributes on an element specified by tag.
- `script(src_name)`: Imports a JavaScript script by creating an element.
- `Jaspy()`: Calls and loads the core `jaspy.js` script file.

Text, Class & ID Management

- `Add_Text(id, a): Appends text inside an element specified by ID.`
- `Add_Text_ByClass(clas, isa, a): Appends text inside an element specified by class.`
- `Add_Text_ByTag(tg, isa, a): Appends text inside an element specified by tag.`
- `Add_ClassByTag(tg, isa, cln) /  Add_ClassById(id, cln): Assigns a class name.`
- `Add_IdByTag(tg, isa, idn) /  Add_IdByClass(cln, isa, idn): Assigns an ID.`

Templates & Export Functions

- `create_landing_page(pageTitle, description, colors): Generates a complete landing page (Logo, Title, Description, Custom Colors).`
- `create_portfolio(name, projects): Generates a portfolio page featuring card layouts for each project.`
- `create_business_website(business_name, services): Generates a business website showcasing a list of services.`
- `export_website(file_name): Clones the entire page, converts images to Base64 data URLs, and exports it as a standalone, downloadable HTML file.`

JASPY Web

Additional main.js functions

This addendum documents three reusable functions now available in **main.js**. They build on JASPY Web's dynamic element helpers and can be called from any page script after main.js has loaded.

Model_portfolio(name, role, location, email, avatar, social)

Builds the professional portfolio interface. The function receives the profile data directly, creates the sidebar and content sections, adds responsive styling, and activates navigation behavior.

Parameter	Purpose
name	Full name displayed in the sidebar and footer.
role	Professional title displayed below the name.
location	Profile location; available to portfolio content or future extensions.
email	Used by the contact button to generate a mailto link.
avatar	URL or static path for the profile portrait.
social	Array of objects such as { label, url, icon }.

Example

```
Model_portfolio( "Ando Tiana", "Python & JavaScript Developer", "Madagascar", "hello@example.com",
"/static/img/ando-tiana.jpg", [{ label: "GitHub", url: "https://github.com/andotiana4", icon: "github"
}] );
```

Navigation and content helpers

create_navigator(containerId, brandName, links)

Creates a horizontal navigation bar inside an existing element. The **links** value must be an array of objects using the properties **text** and **url**. The function creates a brand heading plus one anchor for every link.

```
create_navigator( "root", "JASPY Web", [ { text: "Accueil", url: "#home" }, { text: "Portfolio", url: "#portfolio" }, { text: "Contact", url: "#contact" } ] );
```

create_section(parentId, sectionId, titleText, descriptionText)

Creates a semantic **section** inside an existing parent element. It adds a title and a descriptive paragraph, then applies the standard JASPY Web spacing and title treatment. Use it to compose landing pages, portfolio sections, or service pages.

```
create_section( "root", "about", "About", "I develop web applications with Python and JavaScript." );
```

Implementation notes

- Load **/static/src/jaspy/main.js** before calling these functions.
- The target parent element must exist before calling **create_navigator** or **create_section**.
- Use a unique id for each created section.
- For Model_portfolio, pass an empty array when no social links are required.